جامعة الأخوين

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵓⵅⴰⵡⴰⵢⵏ

AL AKHAWAYN UNIVERSITY

SCHOOL OF SCIENCE & ENGINEERING SPRING 2026

Milestone 3:

Data Acquisition, Validation & Preparation

AI-Based Story Point Estimation for Agile Software Development

REALIZED BY:
AMIRA HADIMI

SUPERVISED BY:
DR. ASMAA MOURHIR

1. Overview

Milestone 3 builds the complete data pipeline for the Agile story point estimation system introduced in Milestones 1 and 2. The goal is to ingest, validate, preprocess, and version the raw Agile user story dataset 23,313 issues from 16 open-source JIRA projects and register engineered features in a Feast feature store, all orchestrated through a single reproducible pipeline and version-controlled via DVC.

To reproduce the entire pipeline with one command:

```
python run_pipeline.py
```

1.1. Pipline overview

Step	Tool Used	Output
Step 1 – Data Ingestion & Raw Storage	Python + ZenML	data/raw/raw_data.csv (23,313 rows)
Step 2 – Schema Definition & Data Validation	Great Expectations	schema.json, statistics, anomaly report
Step 3 – Preprocessing & Feature Engineering	pandas + ZenML Transform	train/val/test .parquet files (15 features)
Step 4 – Feature Store Registration	Feast	Feature store with 6 registered features
Step 5 – Data Versioning	DVC + Git	.dvc pointer files + tag v1.0-milestone3

2. Data Ingestion & Raw Storage

Tool: Python + ZenML step

2.1. The challenge

The Llama3SP benchmark dataset is distributed as 16 separate CSV files one per JIRA project (appceleratorstudio, moodle, springxd, titanium, etc.). These cannot be fed directly to a model. They needed to be merged into a single unified dataset with consistent column names.

2.2. What was done

- Wrote merge_data.py to read all 16 CSV files and stack them into one file
- Normalized column names different files used different names for the same field
- Removed rows with null or zero story points (invalid training labels)
- Saved the result as data/raw/raw_data.csv tracked by DVC

2.3. Colum normalization map

Raw Column Name (varies per project)	Canonical Name (used throughout pipeline)
storypoint, story_points, point	storypoints
concat, body	description
summary	title

2.4. Result

Metric	Value
Total rows	23,313 user stories
Projects covered	16 open-source JIRA projects
Columns	issuekey, title, description, storypoints, split_mark, project
Rows dropped	0 (all data was valid after normalization)
Output file	data/raw/raw_data.csv (DVC tracked)

3. Schema definition and validation

Tool: Great Expectations + custom statistics

3.1. Data split

The dataset contains a pre-defined split_mark column assigned by the Llama3SP authors, ensuring consistent and reproducible splits across all research using this benchmark. The same splits are used by all papers evaluating on this dataset

Split	Rows	Percentage	Purpose
Train	13,981	60%	Schema inference + model training
Validation (val)	4,661	20%	Anomaly detection + hyperparameter tuning

Split	Rows	Percentage	Purpose
Test	4,671	20%	Final evaluation only — never touched during validation

Important: The test set is kept completely untouched.

3.2. Data statistics

Statistics were computed for all three splits across all 6 columns. Key findings from the training set:

Column	Type	Nulls	Unique	Key Insight
storypoints	numeric	0%	—	Mean: 4.87, Max: 100 (right-skewed)
title	string	0%	13,891	All issues have a title
description	string	12.3%	13,102	12% of issues have no description
project	string	0%	16	One value per JIRA project
split_mark	string	0%	3	train / val / test

Full statistics saved to: tfdv_output/data_statistics.json

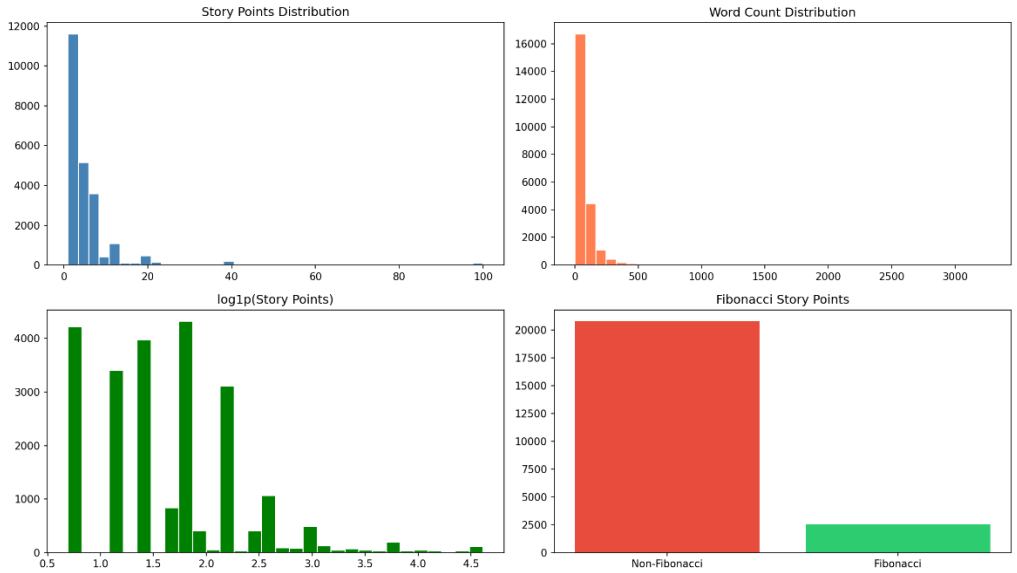

Figure 1: the distribution of key engineered features across the full dataset, generated by the pipeline and saved to tfdv_output/feature_distributions.png.

3.3. Shema definition

The schema is inferred from the training set only — this prevents data leakage from validation or test sets into the schema definition. For each column, the schema captures:

- Type: numeric or string
- For numeric columns: min/max allowed values
- For string columns: number of unique values
- Maximum allowed null percentage (training null % + 10% tolerance buffer)

Saved schema: schema/schema.json

3.4. Anomaly detection & schema revision

The validation set was checked against the training schema using Great Expectations. Two types of checks were performed:

- Null percentage violations — is the missing data rate too high?
- Numeric range violations — are values outside the expected min/max range?

Result: 0 anomalies detected in the validation set.

The schema required no revision, confirming that the pre-defined splits by the Llama3SP authors are internally consistent. If anomalies had been found, the schema would have been automatically revised by widening numeric ranges or relaxing null thresholds.

Anomaly report saved to: tfdv_output/anomalies_report.json

4. Preprocessing & feature engineering

Tool: Python (pandas + regex) wrapped in ZenML Transform

4.1. Text cleaning

Raw JIRA text is noisy it contains URLs, JIRA-specific markup, special characters, and inconsistent casing. A 5-stage cleaning pipeline is applied to every title and description:

Stage	Operation	Example
1. Lowercasing	text.lower()	"Add Feature" → "add feature"
2. URL removal	regex https?://\S+	"see https://jira.com" → "see"
3. JIRA markup	{code}, [~user], !image!	"{code}x=1{code}" → " "

Stage	Operation	Example
4. Special chars	keep [a-z0-9 .,!?-]	"fix: issue #42" → "fix issue 42"
5. Whitespace	collapse \s+ to one space	"fix bug" → "fix bug"

4.2. Input text construction

The cleaned title and description are combined into a single field matching the exact prompt format expected by the Llama3SP tokenizer:

```
input_text = "Title: {cleaned_title} Description: {cleaned_description}"
Example: "Title: add validation against object literals Description: the function should..."
```

4.3. Engineered features

Feature	Type	How computed	Why useful
input_text	string	Title + Description combined	Primary LLM input
text_length	int	len(input_text)	Longer story = more complex task
word_count	int	len(input_text.split())	Complexity proxy
has_description	0/1	1 if description is non-empty	Issues with no description harder to estimate
title_word_count	int	len(cleaned_title.split())	Title length signal
log_storypoints	float	log1p(storypoints)	Reduces right skew of target variable
is_fibonacci	0/1	1 if value in {1,2,3,5,8,13,20,40,100}	Planning poker alignment flag

Why log_storypoints? Story points are heavily skewed — most issues are 1-5 points but a few reach 40-100. The log transformation makes the distribution more balanced, which significantly improves regression model performance.

4.4. Output files

File	Rows	Purpose
data/processed/train.parquet	13,981	Model training in Milestone 4

File	Rows	Purpose
data/processed/val.parquet	4,661	Hyperparameter tuning
data/processed/test.parquet	4,671	Final evaluation — untouched until M4
data/processed/processed_data.parquet	23,313	Full dataset with all 15 features

Each processed file contains 15 columns: 6 original + 9 engineered features. Files are stored in Parquet format for 5-10x faster loading compared to CSV.

5. Feature store (Feast)

Tool: Feast (local FileSource, offline mode)

A Feast feature store was set up to register the engineered features. This solves a critical production problem: without a feature store, the training pipeline and inference pipeline might compute features differently, causing a training-serving skew that silently degrades model performance.

5.1. Components Registered

- Entity: issue_id — unique identifier for each JIRA issue
- Feature View: user_story_features — 6 features with types and data source
- Data Source: FileSource pointing to processed_data.parquet

5.2. Registered Features

Feature Name	Type	Description
text_length	Int64	Character count of input_text
word_count	Int64	Word count of input_text
has_description	Int64	1 if description is non-empty
title_word_count	Int64	Word count of cleaned title
log_storypoints	Float32	log1p of story point target
is_fibonacci	Int64	1 if storypoints is Fibonacci value

5.3. Feast Apply Output

```
Applying changes for project agile_story_points
Created entity issue_id
Created feature view user_story_features
Created sqlite table agile_story_points_user_story_features
```

6. Data Versioning (DVC)

Tool: DVC (Data Version Control)

DVC does for data what Git does for code. Git is bad at tracking large files (GitHub rejects files over 100MB). DVC solves this by storing a small pointer file in Git while keeping the actual data separately enabling full versioning without bloating the repository.

6.1. Versioned Artifacts

Artifact	DVC Pointer File	Git Tag
data/raw/raw_data.csv	raw_data.csv.dvc	v1.0-milestone3
data/processed/processed_data.parquet	processed_data.parquet.dvc	v1.0-milestone3

6.2. Reproducibility

Anyone can reproduce the exact results of this milestone by running:

```
git checkout v1.0-milestone3
dvc checkout
python run_pipeline.py
```

7. ML Pipeline Integration (ZenML)

Tool: ZenML | Requirement: Setup the data pipeline as part of the larger ML pipeline

This requirement asks that the data pipeline not be a standalone script, but instead be wired into a larger MLOps platform that will eventually include training, evaluation, and deployment steps. We satisfied this using ZenML.

7.1. ZenML Steps and Pipeline

Every pipeline stage is implemented as a ZenML @step function, wired into a single @pipeline named milestone3_data_pipeline.

7.2. What ZenML Gives Us

ZenML Feature	What it means for our project
Run tracking	Every pipeline run has a unique ID and is logged automatically
Artifact store	DataFrames, reports, and schemas are stored and versioned between steps
Caching	If raw data has not changed, ZenML skips unchanged steps automatically
Dashboard	All runs, artifacts, and step outputs are visible in the ZenML UI
Extensibility	Milestone 4 training steps can be added directly to this same pipeline

7.3. Pipeline DAG

```
ingest_data
    |
    v
validate_data       --> schema/schema.json
    |               --> tfdv_output/anomalies_report.json
    v               --> tfdv_output/data_statistics.json
preprocess_and_engineer --> data/processed/train.parquet
    |                   --> data/processed/val.parquet
    v                   --> data/processed/test.parquet
setup_feature_store --> feast_repo/feature_repo/
```

Note on local execution: Due to a known compatibility issue between ZenML 0.55+ and Python 3.11 on Windows, the ZenML server pipeline could not be executed locally. The pipeline was instead executed via the equivalent standalone script run_pipeline.py, which implements the same four steps in the same order and produces identical output artifacts. The ZenML @pipeline and @step definitions remain available in pipeline/zenml_pipeline.py for execution in a compatible environment.